

Andromeda Rust Engineering Doctrine 2026

Andromeda Rust Engineering Doctrine 2026

In this article

Executive summary

Cross-check method

Internal Andromeda sources

External sources

1. Rust baseline for 2026

1.1 Baseline decision

1.2 Edition 2024 rules that matter for Andromeda

1.3 Minimum supported Rust version policy

2. Andromeda doctrine mapped to Rust

2.1 Strict boundaries

2.2 Adaptive internals

2.3 Feature acceptance gate

3. Recommended repository architecture

3.1 Root layout

3.2 Root Cargo.toml

3.3 Build profiles

3.4 Cargo config

4. Crate responsibility model

4.1 Core crates

4.2 Language and optimizer crates

4.3 Transaction and storage crates

4.4 Network, security, HA/DR, and operations crates

4.5 Engine facade

5. Folder and module rules

5.1 Standard crate layout

5.2 Module privacy

5.3 Naming

6. Type-driven design

6.1 Newtypes for critical identifiers

6.2 Typestate for transaction lifecycle

6.3 Canonical absence

6.4 Decimal and float discipline

7. Error model

7.1 Error families

7.2 Library vs application errors

7.3 Panic policy

- 8. Unsafe Rust policy
 - 8.1 Default rule
 - 8.2 Unsafe module structure
 - 8.3 Safety comments
 - 8.4 Unsafe review checklist
 - 8.5 Forbidden unsafe patterns
- 9. Binary format and serialization rules
 - 9.1 Canonical binary format
 - 9.2 Persisted format rules
 - 9.3 Codec layout
- 10. Storage and WAL implementation guidance
 - 10.1 WAL first rule
 - 10.2 Page and segment API
 - 10.3 No Rust-native disk structs
 - 10.4 Storage hot/cold policy
- 11. Runtime, async, QUIC, and backpressure
 - 11.1 Async model
 - 11.2 QUIC/RPC rule
 - 11.3 Backpressure
- 12. Hardware-aware Rust
 - 12.1 CPU targets
 - 12.2 SIMD rules
 - 12.3 GPU rules
- 13. DDD for Andromeda in Rust
 - 13.1 Bounded contexts
 - 13.2 Aggregates and procedures
 - 13.3 Value objects
 - 13.4 DDD anti-patterns to reject
- 14. TDD and test strategy
 - 14.1 TDD scope
 - 14.2 Test pyramid adapted to Andromeda
 - 14.3 Crash-test matrix
 - 14.4 Property tests
 - 14.5 Fuzzing targets
 - 14.6 Concurrency tests
 - 14.7 nextest policy
- 15. Observability and diagnostics
 - 15.1 Trace vocabulary
 - 15.2 Rust tracing model
 - 15.3 Required fields
- 16. Cookbook index
 - 16.1 Error handling recipe
 - 16.2 Binary codec recipe
 - 16.3 Async RPC recipe

- 16.4 WAL append recipe
- 16.5 CPU acceleration recipe
- 16.6 GPU batch recipe
- 16.7 Supply-chain recipe
- 17. Documentation rules
 - 17.1 Public API documentation
 - 17.2 ADRs
 - 17.3 Specification template
- 18. CI/CD gates
 - 18.1 Pull request gate
 - 18.2 Nightly safety gate
 - 18.3 Storage/recovery gate
 - 18.4 Performance gate
 - 18.5 Release gate
- 19. Component-specific acceptance checklists
 - 19.1 SRPL compiler
 - 19.2 Procedure contract
 - 19.3 Transaction kernel
 - 19.4 WAL
 - 19.5 Storage
 - 19.6 QUIC/RPC
 - 19.7 Security/IAM
 - 19.8 Optimizer/statistics
- 20. Risk register
- 21. Recommended first vertical slice
 - 21.1 Crates required for first slice
 - 21.2 Tests required for first slice
- 22. Final normative recommendations
- Appendix A - Recommended command cookbook
 - Format and lint
 - Test
 - Miri
 - Fuzz
 - Supply chain
 - Benchmarks
- Appendix B - Source cross-check matrix
- Appendix C - External references
- Appendix D - Internal project references

Andromeda Rust Engineering Doctrine 2026

Architecture, code quality, project structure, test strategy, and low-level implementation guidance

Document status: Consolidated engineering guidance

Audience: Andromeda core engineers, Rust maintainers, database engine architects, security reviewers, performance engineers

Language target: Rust 2024 Edition, Rust stable 1.95.0 baseline unless a formal Architecture Decision Record changes the MSRV

Project target: Andromeda - SGBDRT Moderne 2026

Date: May 7, 2026

In this article

- Define the 2026 Rust baseline for Andromeda.
 - Map Andromeda's doctrine to concrete Rust engineering rules.
 - Specify the recommended workspace, crate, folder, and test structure.
 - Define rules for `unsafe`, binary formats, CPU/GPU acceleration, QUIC/RPC, WAL, storage, and observability.
 - Integrate DDD and TDD without weakening relational, transactional, and deterministic constraints.
 - Provide cookbook-style implementation recipes and quality gates.
 - Cross-check recommendations against Andromeda project sources and current Rust ecosystem sources.
-

Executive summary

Rust should be the primary implementation language for Andromeda because the project requires low-level performance, explicit memory ownership, predictable concurrency, strong type modeling, and narrow `unsafe` boundaries. Rust is not enough by itself. The value comes from using Rust as a disciplined systems language, not as a modern label.

Andromeda should use Rust in 2026 with these rules:

Area	Recommendation
Language baseline	Use Rust 2024 Edition and Rust stable 1.95.0 as the baseline unless an ADR changes the MSRV.
Repository model	Use a strict Cargo workspace with engine crates, contract crates, test harnesses, fuzz targets, tooling, documentation, and supply-chain policy at the root.
Architecture	Match Rust crates to Andromeda engines and planes. Do not create a <code>god_engine</code> crate.
<code>unsafe</code>	Allow <code>unsafe</code> only in sealed, reviewed, tested modules. Every unsafe block must have a local <code>SAFETY:</code> explanation and an associated test strategy.
Binary layout	Never serialize Rust native structs directly to disk or network. Use explicit little-endian codecs and versioned formats.
DDD	Use DDD for language, contexts, entities, value objects, aggregates, and invariants. Do not turn relational persistence into object persistence.
TDD	Use TDD for algorithms, parsers, binders, codecs, state machines, and invariants. Use crash tests and property tests for storage, WAL, and recovery.
Tooling	Use <code>cargo fmt</code> , Clippy, nextest, doctests, Miri, sanitizers, fuzzing, proptest, loom, cargo-audit, cargo-deny, and cargo-vet as layered gates.
Hardware	Use CPU feature detection and isolated SIMD kernels. Use GPU only for batch analytics, statistics, and vector workloads outside the commit path.
Operations	Build observability into spans, metrics, audit traces, recovery reports, and decision traces from the first vertical slice.

Important

Andromeda should become modern by making ambiguity smaller. It should not become modern by adding unbounded capabilities.

Cross-check method

This document uses two categories of sources.

Internal Andromeda sources

The recommendations are cross-checked against the consolidated Andromeda documents:

- `00_ANDROMEDA_INDEX_ET_MODE_DE_LECTURE.md`
- `01_DOCTRINE_LEXIQUE_ARCHITECTURE_CATALOGUE_MODELIZATION.md`
- `02_TYPE_SYSTEM_SRPL_PROCEDURES_MAPS.md`
- `03_TRANSACTION_WAL_MVCC_STORAGE_RECOVERY.md`
- `04_QUIC_RPC_SECURITY_HADR_OPERATIONS.md`
- `05_OPTIMIZER_STATS_ANALYTICS_HARDWARE_ROADMAP_SOURCES.md`
- `Moteur de stockage sécurisé.txt`
- `Rapport consolidé sur les SGBDR, ACID, les transactions, l'algebre relationnelle, la normalisation, les statistiques et l'indexation.pdf`

- [Corpus de reference pour les SGBDR, ACID, l'algebre relationnelle, la normalisation, les statistiques et les structures de recherche.pdf](#)
- [Fondation de SRPL pour un langage procedural relationnel strict.pdf](#)
- Previous CPU and GPU Rust research notes.

External sources

The recommendations are cross-checked against official or primary sources where available:

- Rust Blog, Rust Edition Guide, Rust Reference, Cargo Book, Rust API Guidelines, Clippy documentation.
- Rust tools: Miri, cargo-fuzz, cargo-nextest, proptest, cargo-deny, cargo-audit, cargo-vet.
- QUIC RFC 9000 and RFC 9001.
- Tokio, rustls, OpenTelemetry Rust documentation.
- Microsoft documentation for DDD and TDD.
- Martin Fowler's test pyramid reference.

1. Rust baseline for 2026

1.1 Baseline decision

Use this baseline for the V0/V1 Andromeda engine:

```
Edition:      Rust 2024
MSRV:         1.95.0 unless changed by ADR
Workspace:    Cargo workspace with resolver = "3"
Targets:      x86_64 and aarch64 first
Build policy: stable Rust for production, nightly only for selected tooling gates
```

Rust 1.95.0 was released on April 16, 2026. It introduced `cfg_select!`, if-let guards in match arms, and several library stabilizations. Use it as the explicit 2026 stable baseline for this document. The baseline is not selected because every new feature is critical. It is selected because Andromeda needs a current, stable, documented, and reproducible toolchain.

1.2 Edition 2024 rules that matter for Andromeda

Rust 2024 Edition is especially relevant because it makes important unsafety boundaries more visible.

Rule	Engineering impact for Andromeda
<code>unsafe extern</code> blocks are required.	FFI to GPU, OS APIs, storage APIs, and cryptographic providers must declare the boundary as <code>unsafe</code> .
Unsafe attributes require <code>unsafe(...)</code> .	<code>no_mangle</code> , <code>export_name</code> , and <code>link_section</code> must be documented and reviewed.
<code>unsafe_op_in_unsafe_fn</code> warns by default.	Unsafe functions no longer hide internal unsafe operations. Use explicit unsafe blocks inside unsafe functions.
<code>edition = "2024"</code> implies resolver 3.	Use Rust-version-aware dependency resolution in the root workspace.
Newly unsafe standard functions are visible.	Avoid global process environment mutation after multi-threaded startup.

Note

Treat Edition 2024 as a governance improvement. It does not eliminate the need for an unsafe policy, but it forces more safety intent into source code.

1.3 Minimum supported Rust version policy

Define the MSRV in a machine-readable place:

```
[workspace.package]
edition = "2024"
rust-version = "1.95"
```

Use the following policy:

Rule	Requirement
MSRV change	Requires an ADR, release note, CI update, and dependency audit.
Nightly use	Allowed only for tooling gates such as fuzzing or sanitizer runs.
Production binaries	Built from stable unless a signed exception exists.
Dependency MSRV	Must be compatible with the workspace MSRV or explicitly isolated.
Custom targets	Do not depend on unstable custom target JSON for production.

2. Andromeda doctrine mapped to Rust

The Andromeda doctrine is "strict at the boundaries, adaptive inside." In Rust, this becomes a concrete set of type, crate, module, and tooling boundaries.

2.1 Strict boundaries

Andromeda boundary	Rust implementation rule
Type System	Use domain newtypes, checked constructors, sealed state transitions, and explicit serialization.
Procedure Contract	Represent contracts as immutable typed descriptors. Hash canonical forms, not source text.
Catalog	Use versioned catalog descriptors and append-only change records.
Transaction	Model transaction states with explicit enums and tpestate where practical.
WAL	Use explicit record codecs, checksums, chain hashes, LSN monotonic checks, and crash tests.
Network	Use typed RPC frames. Do not pass dynamic maps of strings as the main protocol model.
Security	Separate certificate identity, user principal, permissions, and policies in types.
Import	Apply <code>DefinitionBatch</code> through a validated dependency graph and transactional catalog changes.
Audit	Emit structured events. No critical operation without trace IDs.
Recovery	Every persisted effect must be replayable, reconstructible, or explicitly excluded from truth.

2.2 Adaptive internals

Adaptive zone	Rust implementation rule
Optimizer	Use strategy traits and bounded plan classes. Record <code>DecisionTrace</code> .
Plan Cache	Key by <code>ProcedureId</code> , <code>ContractHash</code> , <code>CatalogVersion</code> , <code>StatsVersion</code> , <code>PolicyVersion</code> , and input shape.
Statistics	Publish <code>StatsVersion</code> through candidate validation and controlled switch.
Hardware	Use runtime feature detection and fallback kernels.
Maps	Enforce refresh policies in types and runtime budgets.
Backpressure	Use typed resource budgets and cancellation-safe async boundaries.

2.3 Feature acceptance gate

A new Rust feature, crate, subsystem, or optimization must pass this gate:

Criterion	Required question	Reject or sandbox when
Definability	Can its semantics be stated precisely?	The behavior depends on hidden runtime state.
Determinism	Same input, state, catalog, stats, and policy produce the same result?	It depends on unseeded randomness or ambient order.
Typing	Can the shape be known before execution?	It returns dynamic records or shape-shifting values.
Boundedness	Can CPU, memory, I/O, and temp usage be limited?	It can run unbounded in the core engine.
Observability	Can its effect be measured and traced?	It changes behavior without counters or traces.
Recovery	Can it be replayed or reconstructed after crash?	It mutates durable state outside WAL.
Security	Can IAM and audit constrain it?	It bypasses permission or surface rules.
Versioning	Can it be attached to a version?	It changes semantics without catalog or policy versioning.
Explainability	Can a post-mortem explain the decision?	It is a black-box forced decision.
Disablement	Can it be disabled without corrupting state?	It changes commit or storage truth.

3. Recommended repository architecture

Use a single Cargo workspace with clear crates. Avoid a monolithic engine crate and avoid a crate per file/object. Each crate should have one primary responsibility.

3.1 Root layout

```

andromeda/
  Cargo.toml
  Cargo.lock
  rust-toolchain.toml
  rustfmt.toml
  deny.toml
  supply-chain/
    config.toml
    audits.toml
    imports.lock
  .cargo/
    config.toml
  .config/
    nextest.toml
  docs/
    adr/
    architecture/
    specifications/
    runbooks/
    testing/
  crates/
    andromeda-core/
    andromeda-types/

```

```
andromeda-codec/  
andromeda-contract/  
andromeda-catalog/  
andromeda-srpl/  
andromeda-srpl-parser/  
andromeda-srpl-binder/  
andromeda-srpl-ir/  
andromeda-optimizer/  
andromeda-procedure-store/  
andromeda-transaction/  
andromeda-wal/  
andromeda-storage/  
andromeda-statistics/  
andromeda-analytics/  
andromeda-rpc/  
andromeda-quic/  
andromeda-security/  
andromeda-hadr/  
andromeda-admin/  
andromeda-observability/  
andromeda-engine/  
andromeda-cli/  
tools/  
  xtask/  
  schema-gen/  
  catalog-diff/  
  wal-dump/  
  page-dump/  
  crash-runner/  
tests/  
  integration/  
  recovery/  
  rpc/  
  srpl/  
  storage/  
  fixtures/  
fuzz/  
  Cargo.toml  
  fuzz_targets/  
benches/  
  wal/  
  storage/  
  optimizer/  
  srpl/  
examples/  
scripts/  
ci/
```

3.2 Root Cargo.toml

```
[workspace]  
resolver = "3"  
members = [
```

```

    "crates/*",
    "tools/xtask",
    "tools/schema-gen",
    "tools/catalog-diff",
    "tools/wal-dump",
    "tools/page-dump",
]
exclude = ["fuzz"]

[workspace.package]
edition = "2024"
rust-version = "1.95"
license = "Proprietary"
repository = "https://example.invalid/andromeda"

[workspace.dependencies]
anyhow = "1"
thiserror = "2"
bytes = "1"
tracing = "0.1"
tracing-subscriber = "0.3"
opentelemetry = "0.30"
proptest = "1"

[workspace.lints.rust]
unsafe_op_in_unsafe_fn = "deny"
missing_docs = "warn"
unreachable_pub = "warn"

[workspace.lints.clippy]
correctness = "deny"
suspicious = "deny"
perf = "warn"
complexity = "warn"
style = "warn"

```

Caution

Do not make `[workspace.dependencies]` a dumping ground. Shared dependencies are acceptable only when they are intentionally standardized across crates.

3.3 Build profiles

```

[profile.dev]
opt-level = 0
debug = true
debug-assertions = true
overflow-checks = true
incremental = true

[profile.release]
opt-level = 3
lto = "thin"

```

```
codegen-units = 8
panic = "abort"
strip = "debuginfo"
debug-assertions = false
overflow-checks = false
incremental = false

[profile.release-with-checks]
inherits = "release"
overflow-checks = true
debug-assertions = true

[profile.bench]
inherits = "release"
debug = true
strip = "none"
```

Use `release-with-checks` for pre-production soak tests and crash scenarios. Use production `release` only after invariant-heavy validation.

3.4 Cargo config

```
# .cargo/config.toml
[build]
rustflags = [
    "-C", "force-frame-pointers=yes",
]

[env]
RUST_BACKTRACE = "1"
```

Do not set `target-cpu=native` globally. Use runtime CPU dispatch and policy-driven hardware profiles.

4. Crate responsibility model

4.1 Core crates

Crate	Responsibility	Must not do
andromeda-core	Core clocks, IDs, accounting, feature flags, common primitives.	No storage, catalog, network, or procedure logic.
andromeda-types	Domain primitives, IDs, LSNs, timestamps, policy IDs, version IDs.	No I/O and no engine logic.
andromeda-codec	Canonical binary codecs, endian primitives, checked readers/writers.	No object semantics or transaction decisions.
andromeda-contract	Procedure contracts, shapes, <code>ContractHash</code> , compatibility descriptors.	No plan choice or RPC transport.
andromeda-catalog	Catalog descriptors, <code>DefinitionBatch</code> , dependency graph, versioning.	No direct client API surface.

4.2 Language and optimizer crates

Crate	Responsibility	Must not do
andromeda-srpl	Public SRPL compiler facade.	No persistence.
andromeda-srpl-parser	Lexer, parser, syntax diagnostics.	No semantic binding.
andromeda-srpl-binder	Name, type, cardinality, effect, and policy binding.	No physical plan execution.
andromeda-srpl-ir	Stable semantic IR/ALT.	No source-text dependence.
andromeda-optimizer	Physical plan candidates, costing, plan class, decision trace.	No WAL commit decision.
andromeda-procedure-store	Invocation history, runtime metrics, plan feedback.	No unilateral plan forcing.

4.3 Transaction and storage crates

Crate	Responsibility	Must not do
andromeda-transaction	Transaction state machine, MVCC visibility, commit/rollback protocol.	No raw file parsing outside storage APIs.
andromeda-wal	WAL record types, segment writer/reader, CRC/hash chain, replay iterator.	No business logic.
andromeda-storage	BufferPool, pages, hot/cold segments, manifests, recovery manager.	No Procedure contract parsing.
andromeda-statistics	Histograms, row counts, skew, StatsVersion, publication.	No C5 learned-only behavior.
andromeda-analytics	Column chunks, maps analytics, GPU batch dispatch.	No commit, WAL, rollback, or recovery path.

4.4 Network, security, HA/DR, and operations crates

Crate	Responsibility	Must not do
andromeda-rpc	Frame header, typed RPC messages, structured payloads, result streams.	No QUIC-specific transport policy.
andromeda-quic	QUIC integration, stream lifecycle, flow control, backpressure.	No business procedure semantics.
andromeda-security	mTLS identity mapping, IAM, permissions, policies, audit security events.	No storage truth decisions.
andromeda-hadr	Replication metadata, quorum, fencing, manifests, WAL shipping.	No multi-primary V0.
andromeda-admin	Definition import, maintenance, backup/restore orchestration, debug snapshots.	No unaudited bypass.
andromeda-observability	Tracing, metrics, audit event schemas, recovery reports.	No control decisions without policy input.

4.5 Engine facade

andromeda-engine is the composition root. It wires engines together and owns lifecycle. It must remain thin.

Allowed:

```
configuration
component wiring
startup and shutdown sequencing
service lifecycle
surface registration
health orchestration
```

Forbidden:

```
business rules
storage page parsing
planner algorithms
security bypass
hidden global mutable state
```

5. Folder and module rules

5.1 Standard crate layout

```
crates/andromeda-wal/
Cargo.toml
README.md
src/
  lib.rs
  error.rs
  record.rs
  lsn.rs
  codec.rs
  segment/
    mod.rs
    header.rs
    writer.rs
    reader.rs
    trailer.rs
  replay/
    mod.rs
    iterator.rs
    validation.rs
  test_support.rs
tests/
  wal_segment_roundtrip.rs
  wal_replay_truncated_tail.rs
benches/
  wal_append.rs
```

Rules:

- Keep `lib.rs` as a map, not as implementation.
- Use `error.rs` for public crate-level error types.
- Use `test_support.rs` only behind `#[cfg(any(test, feature = "test-support"))]`.
- Put integration tests in crate-local `tests/` when they test public API.
- Put cross-engine tests under workspace-level `tests/`.

5.2 Module privacy

Use strict module privacy:

```

mod segment;
mod replay;
mod codec;

pub use record::{WalRecord, WalRecordType};
pub use lsn::Lsn;
pub use segment::{WalSegmentReader, WalSegmentWriter};

```

Do not expose internal structs because tests need them. Prefer test-only public helpers with explicit feature gates.

5.3 Naming

Use Rust naming conventions and Andromeda vocabulary together:

Concept	Rust name style	Example
Crate	kebab-case	andromeda-procedure-store
Module	snake_case	procedure_store
Type	UpperCamelCase	ProcedureContract
ID newtype	UpperCamelCase	CatalogVersion, StatsVersion, ContractHash
Constant	SCREAMING_SNAKE_CASE	PAGE_SIZE_16K
Error code	UpperCamelCase enum variant	CardinalityMismatch
Trace name	UpperCamelCase	PlanDecisionTrace

Use `Uuid`, not `UUID`; use `Lsn`, not `LSN`, unless the project intentionally defines an acronym style exception.

6. Type-driven design

6.1 Newtypes for critical identifiers

```

#[derive(Clone, Copy, Debug, Eq, PartialEq, Ord, PartialOrd, Hash)]
pub struct Lsn(u64);

impl Lsn {
    pub const ZERO: Self = Self(0);

    pub fn new(value: u64) -> Self {
        Self(value)
    }

    pub fn get(self) -> u64 {
        self.0
    }
}

```



```

pub fn next(self) -> Option<Self> {
    self.0.checked_add(1).map(Self)
}
}

```

Use newtypes for:

```

DatabaseId
NamespaceId
ObjectId
ProcedureId
InvocationId
TxId
Lsn
CatalogVersion
StatsVersion
PolicyVersion
ContractHash
PlanId
SnapshotId
SegmentId
PageId

```

Do not pass raw `u64` or `Uuid` across engine boundaries when a semantic type exists.

6.2 Typestate for transaction lifecycle

```

pub struct Transaction<S> {
    tx_id: TxId,
    state: S,
}

pub struct Created;
pub struct Active;
pub struct Committing;
pub struct Committed;
pub struct RollingBack;
pub struct RolledBack;

impl Transaction<Created> {
    pub fn activate(self) -> Transaction<Active> {
        Transaction { tx_id: self.tx_id, state: Active }
    }
}

```

Use typestate for transitions that are stable and limited. Use runtime state machines when the transitions depend on I/O failure, async cancellation, or recovery state.

6.3 Canonical absence

Use `Option<T>` internally where absence is local and obvious. Use explicit domain enums when absence has business or protocol meaning.

```
pub enum CustomerLookup {  
    Found(CustomerRecord),  
    NotFound,  
}
```

Do not map SRPL `optional one` into unchecked `T`. The compiler and contract layer must force branch handling.

6.4 Decimal and float discipline

Type family	Use	Rust rule
Exact decimal	Money, accounting, tax, exact quantities.	Use a controlled decimal representation with explicit precision and scale.
Integer	IDs, counts, LSNs, page sizes, offsets.	Use checked arithmetic for persisted offsets and sizes.
Float	Statistics, scores, measurements, vector analytics.	Never use as key, FK, WAL consensus value, or exact invariant.

Use explicit NaN and rounding policy for any float stored in catalog, statistics, or analytical maps.

7. Error model

7.1 Error families

Model errors as typed families:

Family	Rust representation	Example
Business error	Procedure result error enum	<code>InsufficientStock</code>
Contract error	<code>ContractError</code>	<code>ContractHashMismatch</code>
Permission error	<code>SecurityError</code>	<code>ExecuteProcedureDenied</code>
Cardinality error	<code>CardinalityError</code>	<code>ExpectedOneFoundMany</code>
Constraint error	<code>ConstraintError</code>	<code>UniqueViolation</code>
Resource error	<code>ResourceError</code>	<code>TempBytesQuotaExceeded</code>
Transaction error	<code>TransactionError</code>	<code>SerializationFailure</code>
Storage error	<code>StorageError</code>	<code>WalRecordCrcMismatch</code>
System error	<code>SystemError</code>	<code>IoFailure</code>

7.2 Library vs application errors

Use this rule:

Context	Recommendation
Library crate public API	Use typed errors with <code>thiserror</code> or manual <code>Error</code> impls.
CLI, maintenance tools, tests	Use <code>anyhow</code> with context.
Engine runtime	Use typed errors and convert to RPC error families at the boundary.
Audit trace	Store error code, family, severity, trace ID, retry policy, and message.

```
#[derive(Debug, thiserror::Error)]
pub enum WalError {
    #[error("WAL record CRC mismatch at LSN {lsn:?}")]
    CrcMismatch { lsn: Lsn },

    #[error("WAL record length is invalid: {length}")]
    InvalidLength { length: u32 },

    #[error("I/O failure while reading WAL segment {segment:?}")]
    Io {
        segment: SegmentId,
        #[source]
        source: std::io::Error,
    },
}
```

7.3 Panic policy

Do not use panic as an error mechanism in server-critical paths.

Allowed:

```
unit tests
property tests
fuzz harness assertions
compile-time impossible branches with proof
startup validation before accepting traffic
```

Forbidden:

```
WAL append path
commit visibility path
recovery replay path
RPC request handling
security authorization path
storage page parser on untrusted bytes
```

Use `panic = "abort"` for production binaries only after all critical errors are typed and surfaced correctly.

8. Unsafe Rust policy

8.1 Default rule

`unsafe` is allowed only where it materially improves correctness, performance, or interoperability and cannot be reasonably expressed in safe Rust.

Expected unsafe zones:

```
binary page layout parsing
unaligned or raw byte access when justified
mmap or direct I/O wrappers
FFI to OS, crypto, QUIC, GPU, or acceleration libraries
SIMD intrinsics
lock-free structures
custom allocation
atomic memory ordering internals
```

8.2 Unsafe module structure

```
src/page/
mod.rs          // public safe API
raw.rs          // private unsafe primitives
codec.rs        // checked serialization
validation.rs   // invariant checks
tests.rs        // unit tests
```

`raw.rs` must be private unless the crate is specifically a low-level primitive crate.

8.3 Safety comments

Every unsafe block must include a local `SAFETY:` comment.

```
pub fn read_u64_le(input: &[u8], offset: usize) -> Result<u64, CodecError> {
    let end = offset.checked_add(8).ok_or(CodecError::OffsetOverflow)?;
    let bytes = input.get(offset..end).ok_or(CodecError::UnexpectedEof)?;

    let mut fixed = [0_u8; 8];
    fixed.copy_from_slice(bytes);
    Ok(u64::from_le_bytes(fixed))
}
```

Prefer safe code when it is clear and fast enough. The example above should not use `unsafe`.

Unsafe example:

```
pub unsafe fn page_from_aligned_ptr<'a>(ptr: *const u8) -> &'a [u8; PAGE_SIZE] {
    // SAFETY:
    // - The caller guarantees that `ptr` is non-null.
```

```
// - The caller guarantees that `ptr` is aligned for `[u8; PAGE_SIZE]`.
// - The caller guarantees that `ptr..ptr+PAGE_SIZE` is a live allocation.
// - The returned reference does not outlive the mapped page pin.
unsafe { &*(ptr.cast::<[u8; PAGE_SIZE]>()) }
```

8.4 Unsafe review checklist

Check	Required evidence
Boundary	The unsafe API is private or sealed behind safe constructors.
Preconditions	Documented in a <code># Safety</code> section for unsafe functions.
Invariants	Stated in module docs or type docs.
Miri	Applicable tests pass under Miri.
Fuzz	Byte parsers and decoders have fuzz targets.
Sanitizers	Nightly sanitizer job covers the path when feasible.
Property tests	Roundtrip and invalid-input properties exist.
Audit	Code review has an unsafe-specific approval.
Telemetry	Runtime failure mode is observable when relevant.

8.5 Forbidden unsafe patterns

```
transmute Rust structs to disk bytes
reinterpret untrusted bytes as typed structs without validation
repr(Rust) on persisted or network formats
usize/isize in persisted or network formats
raw enum discriminants on disk without versioned codec
unchecked pointer arithmetic without bounds proof
creating references to packed fields
using target-feature code without dispatch guard
using static mut for shared runtime state
calling std::env::set_var after multi-threaded startup
```

9. Binary format and serialization rules

9.1 Canonical binary format

Use explicit little-endian encoders and decoders.

```
u8/u16/u32/u64/u128: little-endian
signed integers: little-endian two's complement representation
bool: u8 with 0 or 1 only
string: length-prefixed UTF-8 with validation
```

bytes: length-prefixed or fixed-length by schema
optional: explicit tag, never nullable implicit payload

9.2 Persisted format rules

Rule	Requirement
Magic	Every persisted file, segment, page, and WAL segment has a magic value.
Version	Every persisted format has major/minor and feature flags.
Bounds	Every length and offset is checked before allocation or read.
CRC/hash	Every persisted block has integrity metadata according to its criticality.
LSN	Every WAL-covered page has a PageLSN or equivalent.
Canonicalization	Hashes are computed over canonical forms, not debug formatting.
Alignment	Never require natural Rust alignment for on-disk byte parsing.

9.3 Codec layout

```
pub trait Encode {  
    fn encoded_len(&self) -> usize;  
    fn encode(&self, out: &mut Vec<u8>) -> Result<(), CodecError>;  
}  
  
pub trait Decode<'a>: Sized {  
    fn decode(input: &'a [u8]) -> Result<Self, CodecError>;  
}
```

For hot paths, provide a checked cursor type:

```
pub struct DecodeCursor<'a> {  
    input: &'a [u8],  
    offset: usize,  
}
```

Do not allocate while parsing fixed-size headers.

10. Storage and WAL implementation guidance

10.1 WAL first rule

The transaction kernel and storage engine must enforce:

Commit visible = durable WAL

Rust implementation requirements:

Requirement	Rust rule
Durable append	WAL writer owns fsync/group-commit policy and exposes typed durability result.
Replayable records	WAL records have deterministic decode and validation.
Chain validation	LSN, previous LSN, CRC, and chain hash are checked during replay.
Truncated tail	Replay stops at last valid record and reports recovery status.
Mid-log corruption	Enter forensic or restore path according to policy.

10.2 Page and segment API

Use safe wrappers:

```
pub struct Page<'a> {
    bytes: &'a [u8],
    page_id: PageId,
}

pub struct PageMut<'a> {
    bytes: &'a mut [u8],
    page_id: PageId,
    dirty: bool,
}
```

Expose mutation through methods that update dirty state and PageLSN obligations.

```
impl<'a> PageMut<'a> {
    pub fn write_slot(&mut self, slot: SlotId, row: &[u8]) -> Result<(), PageError> {
        // checked bounds, slot directory update, checksum invalidation
        self.dirty = true;
        Ok(())
    }
}
```

10.3 No Rust-native disk structs

Do not write this:

```
#[repr(C)]
struct PageHeader {
    magic: u64,
    page_id: u64,
    flags: u16,
}

// Forbidden: writing raw struct bytes to disk.
```

Use explicit codecs:

```
pub struct PageHeader {
    pub magic: [u8; 8],
    pub format_version: u16,
    pub page_id: PageId,
    pub page_lsn: Lsn,
    pub flags: PageFlags,
}
```

10.4 Storage hot/cold policy

Storage tier	Rust implementation note
RAM/BufferPool	Use pin guards and explicit dirty tracking. RAM is never truth.
NVMe WAL	Use P0 I/O priority and bounded group commit. WAL is recent durable truth.
HotStore	Prefer copy-on-write/log-structured V0 for easier proof and recovery.
ColdStore	Immutable, append-only, segmented, manifest-backed, sequential.
Temp/Spill	Quota-bound and never truth.
GPU	No authority over durable state.

11. Runtime, async, QUIC, and backpressure

11.1 Async model

Use Tokio for network and administration orchestration. Do not use async as a reason to blur transaction ownership.

Rules:

```
network I/O: async
RPC stream handling: async
admin jobs: async where useful
WAL commit path: explicit durability API, not hidden task fire-and-forget
CPU-bound planning/statistics: spawn blocking or dedicated scheduler
storage page mutation: owned transaction context
```

11.2 QUIC/RPC rule

QUIC is transport. Andromeda RPC is semantics.

Rust crates should reflect this:

```
andromeda-quic = connection, stream, TLS, flow control
andromeda-rpc  = frames, contracts, payloads, result streams, errors
andromeda-contract = procedure contract and shape semantics
```


Do not let QUIC stream states decide procedure semantics.

11.3 Backpressure

Define typed backpressure signals:

```
pub enum BackpressureSignal {  
    ClientSlow { session: SessionId },  
    WallLag { queue_depth: u32 },  
    TempQuota { used: u64, limit: u64 },  
    ReplicaLag { node: NodeId, lag_lsn: u64 },  
    MaintenanceFence,  
}
```

Backpressure actions must be policy-driven:

```
reduce batch size  
spool under quota  
slow producer  
reject new RPC  
close abusive session  
suspend analytics
```

WAL flush and recovery outrank analytics, statistics, GPU jobs, and predictive evidence.

12. Hardware-aware Rust

12.1 CPU targets

Support:

```
x86_64 baseline  
x86_64 AVX2  
x86_64 AVX-512 where policy allows  
AArch64 baseline NEON  
AArch64 SVE/SVE2 where available
```

Use runtime feature detection and policy.

```
pub trait CrcKernel {  
    fn crc64(&self, input: &[u8]) -> u64;  
}
```

Dispatch once, then use function pointers or trait objects:

```
pub fn select_crc_kernel() -> &'static dyn CrcKernel {  
    #[cfg(target_arch = "x86_64")]  
    {
```

```

        if std::is_x86_feature_detected!("sse4.2") {
            return &CRC_SSE42;
        }

        &CRC_SCALAR
    }
}

```

12.2 SIMD rules

Rule	Requirement
Isolation	SIMD kernels live in isolated modules.
Fallback	Every SIMD path has scalar fallback.
Dispatch	Runtime dispatch must guard target-feature functions.
Tests	Same test vectors must run on scalar and accelerated paths.
Benchmark	Acceleration is accepted only with benchmark evidence.
Safety	Intrinsics are unsafe; document preconditions.

12.3 GPU rules

Allowed GPU workloads:

```

histograms
cardinality estimation batches
skew detection
analytical scans
large aggregations over Maps
benchmark scenarios
vector similarity extensions

```

Forbidden GPU workloads:

```

commit
WAL append or flush
rollback
recovery
single-row OLTP lookup
MVCC short visibility checks
security-critical authorization
catalog publication

```

Important

GPU is an accelerator. It is not a transaction participant.

13. DDD for Andromeda in Rust

DDD is useful for Andromeda if it clarifies business language and boundaries. It is harmful if it pushes the engine toward object persistence or hides relational constraints behind mutable object graphs.

13.1 Bounded contexts

Map bounded contexts to namespaces and crate-level domains.

DDD concept	Andromeda concept	Rust implementation
Bounded context	Namespace / subsystem	Crate or module boundary with explicit API.
Ubiquitous language	Native Andromeda lexicon	Type names, descriptors, diagnostics, trace names.
Entity	Table row identity / catalog object	Newtype ID + descriptor struct.
Value object	Domain type	Immutable newtype with checked constructor.
Aggregate	Transactional invariant boundary	Procedure + constraints + write set.
Domain service	Operation not owned by one entity	Procedure logic or compiler/engine service.
Domain event	WAL/audit/event trace	Typed event, versioned and replay-aware when durable.

13.2 Aggregates and procedures

For Andromeda, an aggregate should not be a long-lived in-memory object graph. It should be a transactional invariant boundary expressed through:

```
Procedure contract
+ declared read/write set
+ table constraints
+ map consistency policy
+ transaction isolation policy
+ RowsAffected checks
+ WAL coverage
```

13.3 Value objects

Use value objects for domain-specific primitives:

```
#[derive(Clone, Debug, Eq, PartialEq, Hash)]
pub struct EmailAddress(String);

impl EmailAddress {
    pub fn parse(input: &str) -> Result<Self, EmailAddressError> {
        // validate, normalize, enforce length/collation policy as required
        Ok(Self(input.to_owned()))
    }
}
```

Value objects should be:

```
immutable
validated at construction
cheap to clone when possible
serializable through explicit codecs
independent from storage layout
```

13.4 DDD anti-patterns to reject

Anti-pattern	Why reject it
Active Record engine objects	Blurs persistence, transaction, and domain behavior.
Mutable entity graphs across transactions	Conflicts with MVCC, WAL, and deterministic recovery.
Domain events outside WAL for durable effects	Breaks recovery and forensic traceability.
Repository pattern hiding Procedure contract	Weakens the RPC-only doctrine.
One global domain model	Conflicts with bounded contexts and namespace policies.

14. TDD and test strategy

14.1 TDD scope

Use TDD for:

```
parsers
binders
contract compatibility
catalog dependency graph
WAL record codecs
page/segment codecs
transaction state transitions
optimizer rewrite rules
statistics publication state machine
IAM policy evaluation
RPC frame validation
```

Do not pretend that TDD alone validates:

```
crash recovery
I/O durability
race freedom
unsafe soundness
cluster failover
performance stability
adversarial fuzz input
```

These require additional gates.

14.2 Test pyramid adapted to Andromeda

Layer	Volume	Examples
Unit tests	Very high	Codecs, newtypes, cost formulas, diagnostics.
Property tests	High	Roundtrip, monotonicity, invariants, idempotence.
Fuzz tests	High for parsers/codecs	SRPL parser, RPC frames, WAL records, page headers.
Miri tests	Targeted	Unsafe wrappers, pointer-sensitive code.
Loom tests	Targeted	Atomics, lock-free structures, concurrency protocols.
Integration tests	Medium	Procedure call through RPC local harness.
Crash/recovery tests	Medium, mandatory	Kill process at exact WAL/recovery points.
HA/DR tests	Lower but critical	Quorum, fencing, WAL shipping, promotion.
End-to-end tests	Low	Full user scenario with catalog, procedure, transaction, result stream.
Benchmarks	Continuous but separate	WAL throughput, page scan, optimizer latency, ResultStream.

14.3 Crash-test matrix

Implement a crash runner that can kill the engine at deterministic injection points:

Injection point	Expected result
Before durable <code>TxBegin</code>	Transaction does not exist.
After <code>TxBegin</code> , before mutation WAL	Incomplete transaction ignored or rolled back.
After mutation WAL, before <code>TxCommit</code>	UNDO/rollback.
After durable <code>TxCommit</code> , before client ACK	Transaction is committed after recovery.
After client ACK, before dirty page flush	Transaction is visible after WAL REDO.
During manifest switch	Old or new manifest is valid; never half-published.
During map delta apply	Map consistency follows WAL and refresh policy.

14.4 Property tests

Use proptest for invariants:

```
proptest::proptest! {  
    #[test]  
    fn wal_record_roundtrips(record in any_valid_wal_record()) {  
        let encoded = record.encode_to_vec().unwrap();  
        let decoded = WalRecord::decode(&encoded).unwrap();  
        prop_assert_eq!(record, decoded);  
    }  
}
```

Recommended properties:

```
encode/decode roundtrip
invalid length never panics
LSN ordering is monotonic
catalog dependency sort is stable
contract hash is deterministic
stats publication state machine rejects invalid transitions
page free-space accounting never underflows
```

14.5 Fuzzing targets

Use cargo-fuzz for untrusted or semi-trusted input parsers:

```
fuzz_targets/srpl_parser.rs
fuzz_targets/rpc_frame.rs
fuzz_targets/structured_object.rs
fuzz_targets/wal_record.rs
fuzz_targets/page_header.rs
fuzz_targets/manifest.rs
fuzz_targets/segment_index.rs
```

Fuzz target rule:

```
No panic.
No OOM.
No unbounded allocation.
No UB under sanitizer-supported configurations.
Invalid input returns typed error.
```

14.6 Concurrency tests

Use Loom only for small concurrency models:

```
latch state machine
reference-counted pin guards
lock-free queue internals
atomic flag protocols
backpressure state propagation
```

Do not try to model the whole engine with Loom.

14.7 nextest policy

Use cargo-nextest for workspace test execution:

```
cargo nextest run --workspace
cargo test --doc --workspace
```

Keep doctests separate because nextest does not replace doctest execution.

15. Observability and diagnostics

15.1 Trace vocabulary

Use Andromeda trace names directly:

```
ProcedureInvocationTrace
TransactionTrace
PlanDecisionTrace
CatalogChangeTrace
DefinitionBatchApplyTrace
SecurityAuditTrace
AdminOperationTrace
RecoveryTrace
ClusterEventTrace
```

15.2 Rust tracing model

Use `tracing` for local structured spans and connect to OpenTelemetry at boundaries.

```
#[tracing::instrument(
    name = "wal.append",
    skip_all,
    fields(tx_id = ?tx_id, record_type = ?record_type)
)]
pub fn append_record(&mut self, record: &WalRecord) -> Result<Lsn, WalError> {
    // ...
    Ok(lsn)
}
```

15.3 Required fields

Operation	Required fields
Procedure invocation	InvocationId , ProcedureId , ContractHash , CatalogVersion , PolicyVersion .
Plan decision	PlanId , candidate count, rejected candidates, StatsVersion , cost terms, risk penalty.
WAL append	TxId , Lsn , record type, bytes, sync policy.
Commit	TxId , commit LSN, durability mode, flush latency.
Recovery	snapshot, manifest, WAL start, last valid LSN, redo count, rollback count, open mode.
IAM decision	principal, certificate ID, surface, operation, deny/allow reason, policy version.

16. Cookbook index

This section defines recommended recipes. A recipe is approved only when it has a test gate.

16.1 Error handling recipe

Use when	Recipe
Library API	Use typed error enum.
CLI/tooling	Use <code>anyhow::Result</code> with context.
RPC boundary	Convert typed error into protocol error family.
Procedure business error	Return typed business error, not string-only message.

Acceptance tests:

```
error displays stable code
source chain is preserved
RPC conversion is deterministic
no sensitive internal details leak on Application Surface
```

16.2 Binary codec recipe

Use when	Recipe
WAL/page/manifest/RPC frame	Explicit checked codec.
Fixed header	Decode without allocation.
Large payload	Decode descriptor first, stream payload.
Hashing	Hash canonical encoded form.

Acceptance tests:

```
roundtrip property
malformed input fuzz target
truncated input returns error
large length does not allocate before quota check
```

16.3 Async RPC recipe

Use when	Recipe
QUIC streams	Use async I/O with bounded buffers.
ResultStream	Send metadata before payload.
Slow client	Apply typed backpressure.
Large StructuredObject	Batch and validate descriptors first.

Acceptance tests:

```
client disconnect cancels cleanly
slow client triggers backpressure
```



```
invalid frame returns ProtocolError
metadata precedes payload
```

16.4 WAL append recipe

Use when	Recipe
Transaction mutation	Append record before visibility.
Commit	Durable flush before commit visible.
Group commit	Policy-bound, observable latency.
WAL tail corruption	Stop at last valid record.

Acceptance tests:

```
crash before flush invisible
crash after flush visible after recovery
truncated record does not panic
chain hash mismatch enters forensic policy
```

16.5 CPU acceleration recipe

Use when	Recipe
CRC/hash/compression/scans	Scalar baseline + optional accelerated kernel.
x86_64	Use <code>is_x86_feature_detected!</code> and <code>#[target_feature]</code> .
aarch64	Use target-specific modules and runtime or policy detection where available.
Deployment	Select hardware profile at startup and record it.

Acceptance tests:

```
same vectors on scalar and SIMD
feature-disabled path works
unsafe intrinsic preconditions documented
benchmark demonstrates benefit
```

16.6 GPU batch recipe

Use when	Recipe
Statistics build	GPU candidate, CPU validation, controlled publish.
Analytical map scan	SnapshotOnly or deferred map, batch chunks.
Vector extension	Isolated and non-transactional.
Benchmark scenario	Cancelable and quota-bound.

Acceptance tests:

```
CPU fallback is always available
GPU job cancellation does not affect transaction state
published stats are versioned
commit path never depends on GPU completion
```

16.7 Supply-chain recipe

Use when	Recipe
Every CI run	<code>cargo audit</code> and <code>cargo deny check</code> .
Dependency admission	<code>cargo vet</code> audit or accepted exemption.
License policy	<code>deny.toml</code> at root.
Critical crates	Prefer minimal dependencies and review transitive graph.

Acceptance tests:

```
Cargo.lock audited
licenses allowed
duplicate critical crates reviewed
untrusted sources rejected
vet gaps reported
```

17. Documentation rules

17.1 Public API documentation

Every public crate must include:

```
crate-level docs
purpose
non-goals
safety model if applicable
examples
error model
observability notes
versioning notes
```

Every public unsafe function must include:

```
# Safety
caller obligations
lifetime assumptions
alignment assumptions
aliasing assumptions
```

threading assumptions
panic behavior

17.2 ADRs

Use ADRs for decisions that affect stability, safety, or performance.

Required ADRs for V0:

ADR-0001 Rust baseline and MSRV
ADR-0002 Workspace and crate boundaries
ADR-0003 Unsafe Rust policy
ADR-0004 Canonical binary format and endian policy
ADR-0005 WAL durability and group commit policy
ADR-0006 Page size and segment size defaults
ADR-0007 QUIC/RPC crate boundary
ADR-0008 Observability and trace schema
ADR-0009 GPU exclusion from commit path
ADR-0010 Supply-chain policy

17.3 Specification template

```
# Specification: <Name> v0

## Purpose

## Scope

## Non-goals

## Data structures

## Invariants

## Serialization

## State transitions

## Error model

## Security model

## Observability

## Recovery behavior

## Compatibility

## Tests

## Rejection criteria
```

18. CI/CD gates

18.1 Pull request gate

```
cargo fmt --check
cargo clippy --workspace --all-targets --all-features -- -D warnings
cargo nextest run --workspace --all-features
cargo test --doc --workspace
cargo deny check
cargo audit
```

18.2 Nightly safety gate

```
cargo +nightly miri test -p andromeda-codec
cargo +nightly miri test -p andromeda-wal
cargo +nightly fuzz run wal_record -- -max_total_time=300
sanitizer jobs for selected targets where supported
```

18.3 Storage/recovery gate

```
crash runner matrix
manifest switch interruption tests
WAL truncation tests
page torn write tests
snapshot restore tests
PITR restore tests
```

18.4 Performance gate

```
benchmark controlled hardware profile
record compiler version
record CPU/GPU/NVMe profile
compare against previous baseline
fail on critical regression above threshold
store benchmark evidence in Procedure Store or benchmark registry
```

18.5 Release gate

```
no unreviewed unsafe blocks
no unresolved cargo audit critical/high advisories
no cargo deny policy violations
vet gaps accepted or closed
crash/recovery suite passed
backup/restore test passed
```

forensic startup test passed
release artifacts reproducibly identified

19. Component-specific acceptance checklists

19.1 SRPL compiler

Parser accepts canonical examples.
Parser rejects ambiguous grammar.
Binder resolves names, types, cardinality, and effects.
Optional values require branch handling.
Set semantics are default.
No dynamic SQL text path exists in the core.
IR hash is stable across formatting changes.
Diagnostics have stable codes.

19.2 Procedure contract

ContractHash is deterministic.
Input and output shapes are canonical.
StructuredObject descriptors include layout and row count metadata.
Compatibility is explicit: additive, breaking, deprecated, rejected.
Required permissions are part of the contract.
Read/write sets are declared or inferred with trace.

19.3 Transaction kernel

State transitions are explicit.
Poisoned state cannot continue normal execution.
Commit visible requires durable WAL.
Rollback is possible after controlled failure.
Isolation policy is visible in contract or policy.
Serialization failures are typed and retry-aware.

19.4 WAL

Record type is versioned.
LSN is monotonic.
PrevLsn and chain hash validate.
CRC mismatch is detected.
Truncated tail handling is deterministic.
Replay produces RecoveryReport.
Fuzz target covers record parser.
Crash matrix passes.

19.5 Storage

Page format uses explicit codec.
No Rust-native struct transmute to disk.
HotStore is reconstructible.
ColdStore is immutable after publication.
Manifest is signed or hash-verified according to policy.
SegmentIndex enables startup without full cold scan.
Scrub is low priority and cancelable.

19.6 QUIC/RPC

Application Surface cannot call admin operations.
Frames are typed and bounded.
Metadata precedes payload.
Backpressure is enforced.
Payload length is validated before allocation.
Contract probing is permission-controlled.
Errors do not leak internal details on Application Surface.

19.7 Security/IAM

mTLS certificate is not the logical user.
CertificateIdentity maps to UserPrincipal.
Deny policy wins unless audited break-glass applies.
SurfaceScope is enforced.
Every decision emits SecurityAuditTrace.
Break-glass is bounded, timed, and auditable.

19.8 Optimizer/statistics

Plan uses CatalogVersion, StatsVersion, ContractHash, and PolicyVersion.
DecisionTrace records candidates, costs, stats, and rejection reasons.
Multi-plan cache is bounded.
Statistics publish through candidate validation.
Learned components propose evidence only.
Fallback classic strategy exists.

20. Risk register

Risk	Severity	Rust-specific mitigation
Unsafe unsoundness	Critical	Unsafe policy, Miri, fuzzing, sanitizers, review.
Panic in critical path	Critical	Typed errors, panic audit, release <code>panic = abort</code> .
Native struct serialization	Critical	Explicit codecs and lint/review policy.
Async cancellation corruption	High	Transaction guards, cancellation-safe boundaries, recovery tests.
Plan cache explosion	High	Bounded plan classes and cache quotas.
Learned component overreach	High	Evidence-only path and fallback.
GPU dependency in commit	Critical	Crate boundary and policy rejection.
QUIC/RPC complexity	High	Minimal frame V0 and typed errors.
Dependency supply-chain issue	High	cargo-audit, cargo-deny, cargo-vet.
Test suite too slow	Medium	nextest, profiles, partitioned gates.
Over-modularization	Medium	Crates by engine responsibility, not by every small object.
DDD object-persistence drift	High	Procedures and relational constraints remain canonical.

21. Recommended first vertical slice

The first implementation milestone should prove the critical path, not the broad feature set.

```
Catalog minimal
+ Type System minimal
+ SRPL minimal
+ Procedure contract
+ RPC local harness
+ TransactionScope
+ WAL durable
+ Commit visible
+ ResultStream typed
+ Procedure Store trace
+ crash/recovery test
```

21.1 Crates required for first slice

```
andromeda-core
andromeda-types
andromeda-codec
andromeda-contract
andromeda-catalog
andromeda-srpl-parser
andromeda-srpl-binder
andromeda-srpl-ir
```

```
andromeda-transaction
andromeda-wal
andromeda-storage
andromeda-rpc
andromeda-procedure-store
andromeda-observability
andromeda-engine
```

21.2 Tests required for first slice

```
contract hash golden tests
SRPL parse/bind tests
catalog DefinitionBatch rollback test
RPC local invoke test
WAL append/flush test
commit visibility test
kill-before-flush crash test
kill-after-flush crash test
RecoveryReport validation test
ProcedureInvocationTrace test
```

22. Final normative recommendations

Use Rust as a strict systems language:

1. Use Rust 2024 Edition and a declared MSRV.
2. Use a Cargo workspace that mirrors Andromeda engines and planes.
3. Keep unsafe code small, private, documented, fuzzed, and reviewed.
4. Serialize explicit binary formats. Never persist Rust native layouts.
5. Use DDD for vocabulary and boundaries, not object persistence.
6. Use TDD for design and local correctness, then extend with property, fuzz, Miri, crash, and recovery tests.
7. Keep QUIC transport separate from RPC semantics.
8. Keep GPU outside the commit, WAL, rollback, and recovery path.
9. Make every adaptive decision versioned, bounded, observable, explainable, and disableable.
10. Start with the vertical slice that proves durable commit and deterministic recovery.

Important

The strongest Rust architecture for Andromeda is not the one with the most crates, the most macros, or the most advanced hardware paths. It is the one where every boundary is explicit, every unsafe assumption is reviewable, every durable mutation is recoverable, and every critical decision can be explained after the fact.

Appendix A - Recommended command cookbook

Format and lint

```
cargo fmt --all --check
cargo clippy --workspace --all-targets --all-features -- -D warnings
```

Test

```
cargo nextest run --workspace --all-features
cargo test --doc --workspace
```

Miri

```
cargo +nightly miri test -p andromeda-codec
cargo +nightly miri test -p andromeda-wal
```

Fuzz

```
cargo +nightly fuzz run wal_record -- -max_total_time=300
cargo +nightly fuzz run rpc_frame -- -max_total_time=300
cargo +nightly fuzz run srpl_parser -- -max_total_time=300
```

Supply chain

```
cargo audit
cargo deny check
cargo vet
```

Benchmarks

```
cargo bench --workspace
```

Appendix B - Source cross-check matrix

Source area	Cross-checked conclusion	Rust implementation implication
Andromeda doctrine	Strict boundaries and adaptive internals.	Engine crates and policies must encode boundaries.
SRPL foundation	Strict relational procedure language, explicit absence, explicit cardinality, IR.	Parser, binder, contract, and IR crates must remain separate.
Transaction/WAL/recovery	Durable WAL before visible commit; recovery is C5.	WAL and transaction crates must be tested by crash matrix.
QUIC/RPC/security	QUIC is transport; RPC contracts carry semantics; surfaces are separated.	Separate QUIC, RPC, contract, and security crates.
Optimizer/statistics/hardware	Learned components are evidence, not truth; GPU is outside commit.	Keep learned/GPU paths C0-C2 with fallback.
Storage secure format	Segment, manifest, WAL, HotStore, ColdStore, SegmentIndex.	Use explicit codecs and no native struct serialization.
Rust 2024 Edition	Unsafe boundaries are more visible.	Deny unsafe-op-in-unsafe-fn and document all unsafe.
Cargo workspace/profiles	Workspaces share lockfile, target, profiles, dependency policy.	Root workspace controls lints, profiles, dependencies, supply-chain.
Testing tools	nextest, fuzzing, Miri, proptest, loom support different risk classes.	Use layered quality gates, not a single test runner.
DDD/TDD sources	DDD clarifies bounded contexts; TDD uses failing test, pass, refactor.	Use DDD/TDD where they strengthen boundaries and invariants.

Appendix C - External references

- Rust Blog. "Announcing Rust 1.95.0." <https://blog.rust-lang.org/2026/04/16/Rust-1.95.0/>
- Rust Edition Guide. "Unsafe extern blocks." <https://doc.rust-lang.org/edition-guide/rust-2024/unsafe-extern.html>
- Rust Edition Guide. "Unsafe attributes." <https://doc.rust-lang.org/stable/edition-guide/rust-2024/unsafe-attributes.html>
- Rust Edition Guide. "unsafe_op_in_unsafe_fn warning." <https://doc.rust-lang.org/edition-guide/rust-2024/unsafe-op-in-unsafe-fn.html>
- Rust Edition Guide. "Cargo: Rust-version aware resolver." <https://doc.rust-lang.org/stable/edition-guide/rust-2024/cargo-resolver.html>
- Cargo Book. "Workspaces." <https://doc.rust-lang.org/cargo/reference/workspaces.html>
- Cargo Book. "Profiles." <https://doc.rust-lang.org/stable/cargo/reference/profiles.html>
- Rust Reference. "Behavior considered undefined." <https://doc.rust-lang.org/reference/behavior-considered-undefined.html>
- Rust Reference. "Unsafety." <https://doc.rust-lang.org/reference/unsafety.html>
- Miri. <https://github.com/rust-lang/miri/>
- Rust Fuzz Book. "Fuzzing with cargo-fuzz." <https://rust-fuzz.github.io/book/cargo-fuzz.html>
- Rust Fuzz Book. "Guide." <https://rust-fuzz.github.io/book/cargo-fuzz/guide.html>

- cargo-nextest. <https://nexte.st/>
- Proptest. <https://proptest-rs.github.io/proptest/>
- Proptest state machine testing. <https://proptest-rs.github.io/proptest/proptest/state-machine.html>
- RustSec Advisory Database. <https://rustsec.org/>
- cargo-deny checks. <https://embarkstudios.github.io/cargo-deny/checks/index.html>
- cargo-vet. <https://mozilla.github.io/cargo-vet/>
- Rust API Guidelines. <https://rust-lang.github.io/api-guidelines/checklist.html>
- Rust API Guidelines. "Documentation." <https://rust-lang.github.io/api-guidelines/documentation.html>
- Rust API Guidelines. "Naming." <https://rust-lang.github.io/api-guidelines/naming.html>
- Clippy documentation. <https://doc.rust-lang.org/stable/clippy/>
- Tokio. <https://tokio.rs/>
- rustls. <https://rustls.dev/>
- RFC 9000. "QUIC: A UDP-Based Multiplexed and Secure Transport." <https://www.rfc-editor.org/rfc/rfc9000.html>
- RFC 9001. "Using TLS to Secure QUIC." <https://www.rfc-editor.org/rfc/rfc9001.html>
- OpenTelemetry Rust. <https://opentelemetry.io/docs/languages/rust/>
- Microsoft Learn. "Use tactical DDD to design microservices." <https://learn.microsoft.com/en-us/azure/architecture/microservices/model/tactical-domain-driven-design>
- Microsoft Learn. "Designing a DDD-oriented microservice." <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/ddd-oriented-microservice>
- Microsoft Learn. "Get started with test-driven development using Test Explorer." <https://learn.microsoft.com/en-us/visualstudio/test/quick-start-test-driven-development-with-test-explorer>
- Martin Fowler. "The Practical Test Pyramid." <https://martinfowler.com/articles/practical-test-pyramid.html>
- Rust Cookbook. "Error handling." <https://rust-lang-nursery.github.io/rust-cookbook/errors.html>

Appendix D - Internal project references

- 00_ANDROMEDA_INDEX_ET_MODE_DE_LECTURE.md
- 01_DOCTRINE_LEXIQUE_ARCHITECTURE_CATALOGUE_MODELIZATION.md
- 02_TYPE_SYSTEM_SRPL_PROCEDURES_MAPS.md
- 03_TRANSACTION_WAL_MVCC_STORAGE_RECOVERY.md
- 04_QUIC_RPC_SECURITY_HADR_OPERATIONS.md
- 05_OPTIMIZER_STATS_ANALYTICS_HARDWARE_ROADMAP_SOURCES.md
- Moteur de stockage sécurisé.txt
- Rapport consolidé sur les SGBDR, ACID, les transactions, l'algebre relationnelle, la normalisation, les statistiques et l'indexation.pdf
- Corpus de reference pour les SGBDR, ACID, l'algebre relationnelle, la normalisation, les statistiques et les structures de recherche.pdf
- Fondation de SRPL pour un langage procedural relationnel strict.pdf
- Recherche processeurs et Rust.txt
- Recherche GPU Rust 2026.txt